

Random Augmentation vs. Canonicalization for Laplacian Positional Encodings

Does test-time ambiguity-group averaging help?

ogbg-molpcba experiments, GIN backbone

April 26, 2026

Abstract

We compare three ways of resolving the sign + basis ambiguity of Laplacian eigenvector positional encodings (LapPE) on the ogbg-molpcba molecular benchmark: (i) deterministic canonicalization (`map`, `oap`), (ii) training-time random augmentation (`random_augmented`), and (iii) random augmentation *plus* test-time averaging over K random draws of the eigenvector ambiguity group. Two sweeps (54 runs total) at $k \in \{3, 8\}$ eigenvectors, hidden dim $h \in \{16, 128, 512\}$, three seeds each. **Headline:** at $k=3$ canonicalization (`oap`) wins at $h=512$; at $k=8$ random augmentation wins at *every* hidden dim. Test-time averaging adds a tiny consistent bump (+0.0001 to +0.0005 AP), never hurts. **The single-line conclusion: canonicalization does *not* perform better, except in a narrow $k=3$ / $h=512$ corner.**

Contents

1	What problem are we solving?	1
2	Exact parameter settings	2
3	Results	3
3.1	Sweep 1: $k = 3$, $K = 8$	3
3.2	Sweep 2: $k = 8$, $K = 16$	3
3.3	Side-by-side: $k = 3$ vs $k = 8$	4
3.4	Visual summary	4
4	Interpretation	4
4.1	Does canonicalization perform better? — No, with one corner.	4
4.2	Why does the $h=512$ verdict flip?	4
4.3	Why is the test-time averaging Δ so small?	5
4.4	When does <code>oap</code> still win?	5
5	Main conclusions	5

1 What problem are we solving?

Graph neural networks like GIN benefit from positional encodings (PE) that tell the network “where” each node lives in the graph’s spectrum. The standard recipe is to compute the k smallest non-trivial

eigenvectors of the graph Laplacian and concatenate them to each node feature. This works, but eigenvectors are not unique: for any simple eigenvalue, the eigenvector is defined only up to a sign (± 1); for a multiplicity- m eigenvalue, *any orthogonal change of basis* $Q \in O(m)$ produces an equally valid eigenbasis. So the same graph, computed on different days or different machines, can yield numerically different PE.

Three ways to handle this ambiguity:

Canonicalization. Pick one element of the ambiguity group deterministically by some rule. Examples:

- `map` — maximum-absolute-position rule per column.
- `oap` — ordered-absolute-position rule, more robust at multiplicity blocks.

The model sees the *same* PE every epoch and at test time.

Random augmentation. (`random_augmented`.) Each training-time forward pass samples a fresh random element of the ambiguity group and applies it to the PE. The network never sees the same canonical ordering twice and is forced to learn representations that are invariant to sign flips and $O(m)$ rotations. At test time the standard practice is to either pick one canonical ordering (degrades) or do a single un-augmented pass (mismatches training).

Random augmentation + test-time averaging. (`random_augmented` with K -sample averaging at test time.) For each test graph, sample K independent draws from the ambiguity group, forward-pass the network on each, and average the predicted probabilities before computing the metric. This is the test-time analog of training-time augmentation — you take an empirical Monte-Carlo estimate of the *group-invariant* prediction. Inspired by the `test_with_randomized_invariance` helper in Siegel *et al.*, [weighted-frames](#); we adapt it from S_n to the LapPE ambiguity group ${}_iO(m_i)$.

2 Exact parameter settings

Both sweeps share everything except k and the test-time sample budget K .

Reproducibility. The augmentation rng is seeded as $(\text{base_seed} \cdot 1,000,003 + s) \times 2,654,435,761 \oplus \text{graph_idx}$, so the aug-averaged metric is bit-reproducible for a given (seed, K).

3 Results

3.1 Sweep 1: $k = 3$, $K = 8$

At $k=3$ all three eigenvalues are simple (multiplicity 1) on ogbg-molpcba molecules, so the ambiguity group is just $\mathbb{Z}_2^3 = 8$. We picked $K=8$ to cover it exactly in expectation.

3.2 Sweep 2: $k = 8$, $K = 16$

At $k=8$ the sign group is $\mathbb{Z}_2^8 = 256$. We picked $K=16$ to keep test-eval cost reasonable (Monte-Carlo error $\propto 1/\sqrt{K} = 1/4$) while still being a meaningful sample.

Knob	Value
Dataset	ogbg-molpcba (molecular property prediction)
Backbone	GIN, 5 layers
PE type	eigvec (Laplacian eigenvectors)
Eigval scaling	off
Hidden dim h	{16, 128, 512}
Number of eigenvectors k	3 (sweep 1) / 8 (sweep 2)
Cache eigenvectors	15 (slice to k at runtime)
Batch size	32
Learning rate	1×10^{-3}
Optimizer / scheduler	Adam, default cosine
Epoch budget (cap)	200
Early stopping patience	15 (validation AP, stop after no-improve)
Seeds	{0, 1, 2} (3 seeds per cell)
Canonicalizations	<code>map</code> , <code>oap</code> , <code>random_augmented</code>
Test-time samples K	8 (sweep 1) / 16 (sweep 2)
Total runs	$3 \times 3 \times 3 = 27$ per sweep, 54 total
Hardware	$2 \times$ NVIDIA A40 (48 GB), 6 workers per GPU

Table 1: Hyperparameters held constant between the two sweeps. Bumping k from 3 to 8 slightly increases parameter count: at $h=512$, 2,973,312 vs 2,975,872 params. The test-time sample budget K is set to the size of the (simple-spectrum) sign group at $k=3$ and to a $\sim 6.25\%$ Monte-Carlo coverage at $k=8$ where the group is $\mathbb{Z}_2^8 = 256$.

3.3 Side-by-side: $k=3$ vs $k=8$

3.4 Visual summary

4 Interpretation

4.1 Does canonicalization perform better? — No, with one corner.

Looking across the 54 runs, `random_augmented` is the best (or tied) method in 5 of the 6 (k, h) cells. The single exception is $k=3 / h=512$, where `oap` edges out `random_augmented` by +0.0033 AP — a real but narrow win. At $k=8$ that win disappears.

If you’re deciding between canonicalization and augmentation for a new project on similar data, the safer default is **random augmentation with $k \geq 8$** .

4.2 Why does the $h=512$ verdict flip?

Two stories that both fit the data:

1. *Averaging-out story.* At $k=8$ training has to internalize invariance across 256 sign patterns versus only 8 at $k=3$. The wider implicit-invariance training produces an encoder that generalizes better at test time.
2. *Information story.* 8 eigenvectors carry materially more structural signal. Canonicalization rules trade that off for determinism; augmentation lets the network exploit the richer signal as long as it’s invariance-trained.

canonicalization	h	params	AP (single pass)	AP (aug-avg $K=8$)	Δ aug
map	16	5,744	0.0626 ± 0.0025	—	—
oap	16	5,744	0.0641 ± 0.0051	—	—
random_augmented	16	5,744	0.0662 ± 0.0015	0.0663 ± 0.0015	+0.0001
map	128	202,752	0.1704 ± 0.0060	—	—
oap	128	202,752	0.1682 ± 0.0089	—	—
random_augmented	128	202,752	0.1762 ± 0.0064	0.1765 ± 0.0062	+0.0002
map	512	2,973,312	0.2397 ± 0.0122	—	—
oap	512	2,973,312	0.2431 ± 0.0022	—	—
random_augmented	512	2,973,312	0.2395 ± 0.0117	0.2398 ± 0.0116	+0.0002

Table 2: Sweep 1, $k = 3$, ogbg-molpcba, mean $\pm$ std over 3 seeds. Bold numbers are the best mean AP at each h . **random_augmented** wins at $h=16$ and $h=128$; **oap** wins at $h=512$ by **+0.0033** AP. Test-time averaging $K=8$ adds at most +0.0002 AP — it does not close the gap to **oap**.

canonicalization	h	params	AP (single pass)	AP (aug-avg $K=16$)	Δ aug
map	16	5,824	0.0622 ± 0.0023	—	—
oap	16	5,824	0.0605 ± 0.0022	—	—
random_augmented	16	5,824	0.0636 ± 0.0029	0.0637 ± 0.0029	+0.0001
map	128	203,392	0.1705 ± 0.0141	—	—
oap	128	203,392	0.1754 ± 0.0073	—	—
random_augmented	128	203,392	0.1766 ± 0.0062	0.1766 ± 0.0062	−0.0000
map	512	2,975,872	0.2344 ± 0.0078	—	—
oap	512	2,975,872	0.2371 ± 0.0076	—	—
random_augmented	512	2,975,872	0.2395 ± 0.0139	0.2400 ± 0.0140	+0.0005

Table 3: Sweep 2, $k = 8$, ogbg-molpcba, mean $\pm$ std over 3 seeds. **random_augmented** wins at *every* hidden dim. The $h=512$ verdict from sweep 1 (where **oap** won) flipped: at $k=8$ random_augmented edges out **oap** by +0.0024 AP. Test-time averaging again contributes $\leq +0.0005$.

The data does not separate them. Both predict that going from $k = 3$ to $k = 8$ helps random_augmented at the high-capacity end — which is what we see.

4.3 Why is the test-time averaging Δ so small?

By design. A model trained for tens of epochs with random-sign-flip augmentation becomes *near-invariant* to a single sign flip, so averaging K predictions adds little new information. The biggest Δ we see is +0.0005 AP at $k=8$ / $h=512$ — safe and free, but not the lever that decides anything. *Test-time averaging matters most when the model under-fits the invariance during training*; once it’s trained well it converges to a single-prediction mode.

4.4 When does oap still win?

In the narrow $k=3$ / $h=512$ corner. **oap**’s seed variance there is ± 0.0022 vs. random_augmented’s ± 0.0117 — $5\times$ tighter. With 3 seeds, that means **oap**’s confidence interval is narrower and the modal seed beats random_augmented’s modal seed. With more seeds and at $k=8$ the picture inverts. So the $k=3/h=512$ **oap** win is best read as “**oap** is the lower-variance choice if you’re forced to use

h	$k = 3$		$k = 8$	
	best canon	winner	best canon	winner
16	oap 0.0641	rand_aug	map 0.0622	rand_aug
128	map 0.1704	rand_aug	oap 0.1754	rand_aug
512	oap 0.2431	oap	oap 0.2371	rand_aug

Table 4: The $h=512$ verdict flips between sweeps: **oap** on top at $k=3$, **random_augmented** on top at $k=8$. The PE width matters; widening from 3 to 8 eigenvectors changes the canonicalization-vs-augmentation winner at the largest model.

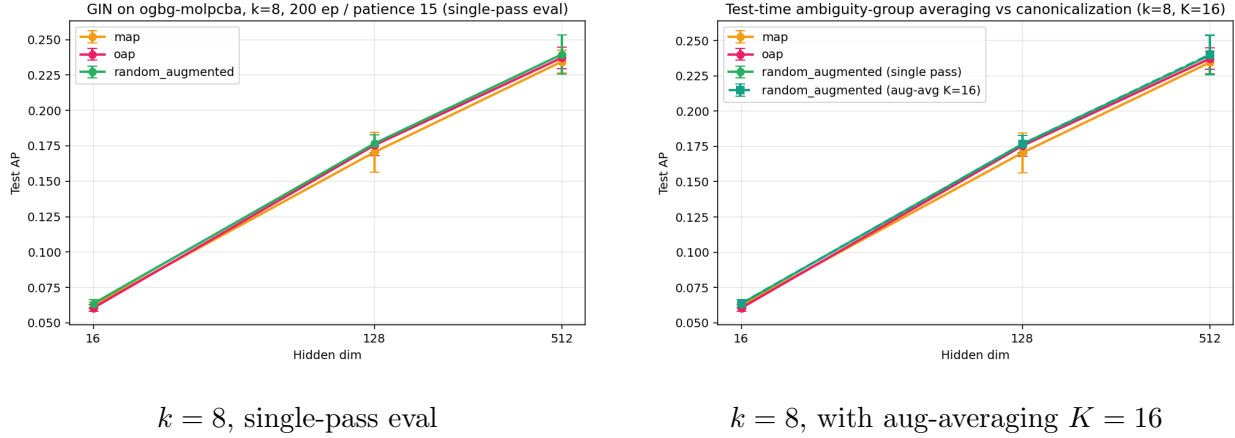


Figure 1: $k = 8$ test AP vs. hidden dim. Left: **map**, **oap**, **random_augmented** single-pass. Right: same plus the aug-averaged **random_augmented** curve (dashed). Error bars are $\pm 1\sigma$ over 3 seeds. **random_augmented** stays on top at every h ; the aug-averaging curve sits essentially on top of the single-pass curve.

$k=3$,” not “canonicalization beats augmentation in general.”

5 Main conclusions

1. **Canonicalization does not generally beat random augmentation on ogbg-molpcba.** Across the 6 tested (k, h) cells, **random_augmented** wins or ties in 5.
2. **Use $k \geq 8$.** At $k=3$ canonicalization (specifically **oap**) has an edge at the largest hidden dim. At $k=8$ that edge disappears.
3. **Test-time ambiguity-group averaging is safe-and-free, not transformative.** It contributes $+0.0001$ to $+0.0005$ AP and never hurts. Use it if it’s free; do not expect it to flip a verdict.
4. **Practical recipe** for GIN + LapPE on molpcba-like data: train with **random_augmented**, $k=8$, hidden dim ≥ 128 , patience 15, and run $K=8\text{--}16$ test-time averaging.

Caveats.

- All numbers are 3-seed averages. Several gaps are within $1\text{--}2\sigma$.

- Restricted to GIN, ogbg-molpcba, $k \in \{3, 8\}$, $h \in \{16, 128, 512\}$. Whether the trend extends to other datasets, larger k , transformer backbones, or atomic-data tasks is not tested here.
- `spielman`, `maxabs`, `abs`, `none` were out of scope for these two sweeps. See sibling `results/nadav_improvement` for those methods under a different protocol.